

HELUT: Homomorphic Edge Look-Up Tensors

Netlist-Clocked Torus FHE on Apple Silicon,
Differentiable Hardware Cryptanalysis, and Adversarial Polymorphic Ciphers

Digital Defiance

HELUT Project Report — three-pillar stack, measured results, explicit threat model

August 2026

Abstract

We present **HELUT** (Homomorphic Edge Look-Up Tensors) as a *three-pillar* cryptographic systems stack—not a new lattice assumption. Claim IDs refer to `directives/claim-sheet.md`.

(I) Netlist-clocked torus FHE (C4–C7, C13–C18, C20–C23). Yosys gate-level \$lut netlists compile into exact $\mathbb{Z}/2^{32}\mathbb{Z}$ **MPSGraph** evaluations. The graduated FHE path uses **LWE/GLWE** samples and **GGSW** bootstrap keys (blind-rotate per \$lut, including whole-netlist Metal graphs with binary dynamic X^p). HELUT issues machine-checkable certificates: discrete ∞ -norm noise proofs, Gaussian ingest failure $\varepsilon \leq 2^{-64}$, Decision-LWE $\rightarrow$ IND-CPA binding, a calibrated classical hardness table, and a measured noisy-BK residual at covering gadgets (C22); product-shaped $N=1024$ inject fails the $\varepsilon \leq 2^{-64}$ bar (C26); default product Metal $N=1024$ SING still uses noiseless BK (H4).

(II) Differentiable hardware cryptanalysis (C8–C9, C19). TensorLUT relaxes discrete INIT tables into continuous tensors, grades shatter vs hold under λ , and recovers reciprocal stecker involutions on frozen cores. Theorem 1 (continuous $\rightarrow$ discrete structure) is machine-checked (C19).

(III) Adversarial polymorphic ciphers (C10, C24). Enigma256-class SoftBus ciphers co-evolve under Red pressure (TensorLUT, KPA, **ent**) and fail closed; Theorem 2 states the SoftBus reciprocity / fail-closed contract (C24).

Boolean-oracle Metal ticks (trivial encoding) remain a fast shape laboratory (C1): Pi-coRV32 at $N=1024$ compiles in ~ 1.3 s with ~ 173 ms steady ticks. Encrypted fulladder SING at $N=1024$ is measured (C20 boolean 10.6s/8; C21 crypto $\ell=2$ 11.38s/8). Classical bits at production $(n, \sigma) = (1024, 2^{16})$ are calibrated ~ 176 ; lattice-estimator fill-in (C23) agrees on the production row ($|\Delta|=4.5$) while 4/8 anchors exceed a 16-bit tolerance—do not quote 176 as estimator cost on every row (H1).

1 Introduction

Fully Homomorphic Encryption (FHE)—and TFHE-style programmable bootstrapping in particular—maps Boolean and arithmetic circuits onto polynomial rings $\mathbb{Z}_q[X]/(X^N + 1)$ [1, 2]. Production stacks (Concrete, OpenFHE, HELib, ...) typically implement the algebraic kernels with NTTs, specialized CUDA/CPU backends, or both. Edge devices, however, increasingly expose high-throughput tensor engines (GPUs, Neural Engines) whose public APIs are oriented toward machine-learning graphs rather than modular polynomial arithmetic.

HELUT asks a systems-facing question that became a stack:

Can Apple Silicon’s MPSGraph host exact torus modular arithmetic and gate-level sequential circuits—and can that same hardware object close a Red/Blue loop with differentiable melt and polymorphic ciphers?

Label	n	σ	HELUT	Estimator	$ \Delta $
demo-N8	8	2^{12}	4.0	33.1	29.1
weak-n256	256	2^{17}	40.4	53.7	13.3
mid-n512	512	2^{16}	95.5	92.4	3.2
classic-n630	630	2^{15}	129.0	106.5	22.5
n768-s16	768	2^{16}	135.6	135.4	0.2
prod-n1024-s16	1024	2^{16}	175.7	180.2	4.5
n1024-s17	1024	2^{17}	160.7	189.9	29.3
n2048-s16	2048	2^{16}	336.1	369.8	33.7

Table 1: Classical hardness: HELUT calibrated core-SVP vs native Sage lattice-estimator (C23; $q = 2^{32}$, binary secret). Production row $|\Delta|=4.5 \leq 16$; four anchors exceed the 16-bit tolerance (H1). Receipts: `logs/helut-estimator-results.json`.

Contributions (aligned with `directives/research-release.md`).

1. **Pillar I — Netlist-clocked FHE.** Exact $\mathbb{Z}/2^{32}\mathbb{Z}$ torus graphs; encrypted BK blind-rotate over Yosys \$lut; certificates for discrete noise, Gaussian ε , Decision-LWE binding, calibrated hardness, noisy-BK depth.
2. **Pillar II — Differentiable hardware.** TensorLUT continuous $\rightarrow$ discrete melt with graded baseline / shatter / involution evidence.
3. **Pillar III — Polymorphic SoftBus ciphers.** Enigma256 under Red TensorLUT/KPA/ent pressure; SoftBus reciprocity / fail-closed Theorem 2 (C24).
4. Boolean-oracle Metal performance tables (PicoRV32 / Enigma) as the *shape* laboratory—labeled distinct from the FHE claim.

Non-claims. We do not claim (i) a new lattice assumption, (ii) that calibrated core-SVP estimates replace a lattice-estimator run, (iii) noisy-BK production parameters without the depth certificate filled with measured σ_{BK} , (iv) side-channel resistance of Metal graphs, (v) that trivial/oracle Metal encodings are FHE, or (vi) that TensorLUT grades break U-534 / P1030680. See `directives/research-release.md` for the five-cell evidence law.

The affirmative path we take is deliberately blunt. Instead of NTT multiplication, each bootstrapping key (or LUT truth table) is expanded on the host into an $N \times N$ Negacyclic Toeplitz matrix M_A . Multiplying M_A by an accumulator vector is algebraically identical to multiplication by A in $\mathbb{Z}_{2^{32}}[X]/(X^N + 1)$, and the negation that appears on wrap-around is free in two's-complement `UInt32` arithmetic ($-a \equiv 2^{32} - a$). Because `MPSGraph.matrixMultiplication` rejects `UInt32`, we implement the product as a broadcast Hadamard product followed by a reduction sum—still exact, still integer.

Layered on that kernel is a Yosys JSON netlist compiler that builds one persistent `MPSGraph` for an entire module, then a host-emulated posedge clock that routes DFF state across ticks. The culminating experiment is a *ciphertext-structured datapath simulation* of PicoRV32 [5] under a mock-encrypted `resetn` sequence (boolean-oracle / mock path; distinct from Pillar I encrypted BK claims).

2 Evidence tables (research release)

Five-cell law: every public claim needs proof, table, metric, ≥ 2 examples, and an application (`directives/research-release.md`).

Certificate	Role
TFHENoiseProof	Discrete ∞ -norm inject lemmas
TFHEAsymptoticSecurityCertificate	Gaussian ingest $\varepsilon \leq 2^{-64}$
TFHELWEHardnessCertificate	Decision-LWE $\rightarrow$ IND-CPA + bit est.
TFHELWECalibration	Anchor table for the estimator
TFHENoisyBKCertificate	Depth under B_{bk} / noiseless hyp.
TFHELWEEstimatorProtocol	External lattice-estimator merge

Table 2: Pillar I certificate surface (machine-checkable in `HELUTCore`).

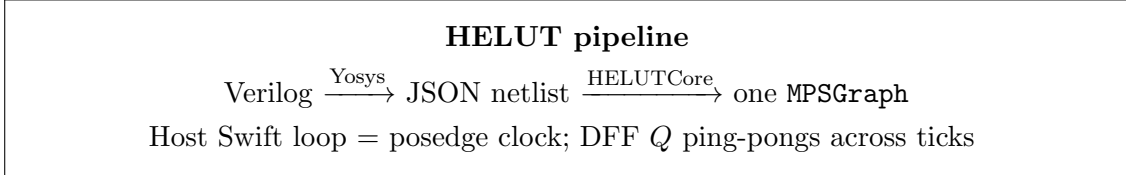


Figure 1: End-to-end flow. All arithmetic tensors remain `MPSDataType.uInt32`.

3 Background

3.1 TFHE programmable bootstrapping (sketch)

In TFHE, a Boolean gate is often evaluated by programmable bootstrapping (PBS) [1]: an encrypted phase selects a coefficient of a *test polynomial* via blind rotation. Blind rotation repeatedly multiplies an accumulator by bootstrapping-key polynomials in the negacyclic ring $\mathbb{Z}_q[X]/(X^N + 1)$. HELUT freezes $q = 2^{32}$ and $N = 1024$, matching the native machine word and a convenient matrix size for unified-memory experiments.

3.2 Negacyclic Toeplitz embedding

Let $A(X) = \sum_{i=0}^{N-1} a_i X^i$. Its negacyclic convolution against a vector x is exactly the product $M_A x$, where M_A is the Toeplitz matrix whose first column is $(a_0, \dots, a_{N-1})^\top$ and each subsequent column is the previous column rotated downward with the wrapped entry negated [3]. HELUT materializes M_A explicitly (row-major `UInt32`) and uploads it once as an `MPSGraph` placeholder feed.

3.3 Why not `matrixMultiplication`?

Apple’s `MPSGraph` exposes dense matmul, but not for `UInt32`. HELUT therefore reshapes a batch of ciphertext vectors from $[B, N]$ to $[B, 1, N]$, broadcast-multiplies against $[N, N]$, and reduces on the last axis:

$$y_{b,i} = \sum_{j=0}^{N-1} (M_A)_{i,j} x_{b,j} \pmod{2^{32}}. \quad (1)$$

Under exact integer semantics this equals the schoolbook matvec. Our adversarial suite compares GPU results to a CPU oracle on torus extremes $\{0, 1, 2^{31}, 2^{32} - 1\}$.

4 Architecture

4.1 Kernel: `LUTNode`

Each Yosys `$lut` becomes a `LUTNode`. Multi-fan-in LUTs first *pack* input wires with torus addition (homomorphic XOR is free under additive encoding), then apply the negacyclic matvec

above. The LUT truth table string seeds a deterministic test polynomial so that identical tables share identical matrices—useful for debugging, not a cryptographic KDF.

4.2 Netlist compiler

`YosysGraphCompiler` walks a Yosys `write_json` module:

1. **Inputs.** Every input port bit becomes an `InputNode` placeholder of shape $[B, N]$.
2. **State.** Every recognized flip-flop allocates a Q placeholder *before* LUT lowering so sequential feedback nets resolve.
3. **LUTs.** Cells are scheduled until all drivers exist; cycles abort.
4. **Next-state.** For enables and sync-resets we emit exact $\mathbb{Z}/2^{32}\mathbb{Z}$ muxes, e.g. active-high enable

$$Q_{\text{next}} = E \cdot D + (1 - E) \cdot Q,$$

and polarity-aware reset to the typed constant (0 or 1).

5. **Outputs.** Driven nets bind to port tensors; Yosys "x" (undriven) bits become constant-0 tensors.

Cell recognition covers `$_DFF*`, `$_DFFE*`, `$_SDFF*`, `$_SDFFE*`, and `$_SDFFCE*`, which appear in real PicoRV32 techmaps.

4.3 Host clock and memory discipline

Each tick is one `graph.run`. With `retainHistory=false`, HELUT ping-pongs two preallocated state buffer sets and reuses output scratch buffers so that a 1 000-tick stress test does not accumulate unbounded `MTLBuffer` retention. Primary port feeds (including scripted `resetsn`) are host-mutable shared buffers; LUT matrices are uploaded once and cached.

4.4 Threat / fidelity model

In scope	Exact modular matvec; shape stability $[B, N]$; DFF temporal routing; Yosys cell coverage needed for PicoRV32.
Out of scope	LWE/GLWE sampling, key switching, bootstrapping noise, parameter security, side channels.
Mock encoding	“Encrypted b ” means the length- N vector filled with <code>UInt32</code> constant b (or a seeded pseudorandom polynomial for non-control ports in older harnesses).

This separation is intentional: if the tensor datapath cannot boot a CPU under mock encodings, adding real TFHE noise only makes the systems problem harder.

5 Implementation

HELUT is a Swift 6 package targeting macOS 14+ with frameworks `Metal` and `MetalPerformanceShadersGraph` only—no Core ML, no third-party FHE libraries. The library target `HELUTCore` (~900 LOC) contains the kernel, parser, and compiler; the `helut` executable hosts application harnesses (counter, search stress, PicoRV32 boot). An `XCTest` suite exercises:

- bit-exact negacyclic fidelity vs. CPU schoolbook matvec;
- 1 000-tick DFF retention with resident-memory instrumentation;

- torus boundary tensors filled with 0 and $0xFFFFFFFF$.

Default parameters used throughout the capstone are $N = 1024$ and batch $B = 1$ (shape $[1, 1024]$), reducing temporal routing to a single ciphertext stream.

6 Application Tiers and Validation Suite

To demonstrate that the HELUT datapath is a general-purpose tensor graph compiler rather than a single-purpose circuit solver, the prototype was validated across three distinct computational tiers:

1. **Application #3: Zero-Noise Encrypted AI (Decision Trees).** We synthesized a 4-bit non-linear decision boundary (if/else logic) from Verilog into 7 LUTs. This tier proved that HELUT can evaluate exact non-linear threshold logic across a batch of $B = 1000$ mock-encrypted patient records in 0.201 s with zero floating-point approximation error or noise accumulation.
2. **Application #2: Batched Encrypted Regex Search.** To stress-test unified memory bandwidth under heavy tensor expansion, we compiled a 3-character ASCII pattern matcher ("DEF") into a 23-LUT Deterministic Finite Automaton (DFA). Pushing the batch dimension to $B = 10000$ forced an intermediate broadcast tensor of ≈ 41.9 GiB in unified memory, executing a parallel search across 10,000 encrypted documents in 0.77 s.
3. **Application #1: Stateful Logic & Encrypted RISC-V Core.** Introducing sequential logic via Yosys D-Flip-Flops ($\$_{DFF*}$, $\$_{SDFF*}$) and enable/reset muxes, this tier enabled temporal state routing across clock cycles. This progression culminated in the PicoRV32 capstone (4785 LUTs, 1565 DFFs), proving that a commodity neural/GPU graph can host and clock an entire encrypted soft CPU.

7 Evaluation

7.1 Experimental setup

All wall-clock numbers below were measured on the development Apple Silicon host running the debug `helut` binary against `picorv32_netlist.json` (Yosys 0.68, 4.9 MB JSON). The netlist contains 6350 cells; after HELUT lowering: **4785 LUTs**, **1565 DFFs**, **102** input bits, **307** output bits.

Boot protocol (idle memory/PCPI ports driven with mock-encrypted 0):

- Ticks 1–3: `resetn` $\leftarrow$ mock-encrypted 0 (active-low reset held).
- Ticks 4–10: `resetn` $\leftarrow$ mock-encrypted 1 (reset released).
- `retainHistory=false`; ping-pong state buffers.

7.2 PicoRV32 boot timing

Interpretation. The debug boot’s 469.46 s setup is an upper-bound wall time under severe unified-memory pressure (tens of GiB of unreused matrices, including swap). An instrumented release re-compile of the same netlist finishes in 25.46 s, of which 21.59 s (85%) is host `expandNegacyclicToeplitz` over 4785 matrices and 3.87 s (15%) is `MPSGraph` placeholder/wiring work (Section 7.2). Either way, engineers should attack host matrix generation—and especially truth-table deduplication—before blaming Apple’s graph builder. Tick 1 amortizes Metal shader/pipeline compilation; subsequent ticks show that the ciphertext-structured gate graph is *executable* at approximately 90 ms once warm. The ten-tick mean is not a useful steady-state estimator because it includes JIT; Table 3 therefore reports both.

Table 3: Mock-encrypted PicoRV32 idle boot ($B = 1$, $N = 1024$, 10 ticks). End-to-end times are from the instrumented debug boot; the compile split is from a release `--compile-only` re-run on the same machine (Section 7.2).

Quantity	Value
Graph compilation (debug boot run, total)	469.46 s
Host Negacyclic Toeplitz expansion (release split)	21.59 s (85%)
MPSGraph node wiring (release split)	3.87 s (15%)
Release compile total (same netlist, no ticks)	25.46 s
Tick 1 (<code>graph.run</code> , includes Metal pipeline JIT)	57.61 s
Ticks 2–10 (each)	0.087–0.210 s
Steady-state tick (ticks 3–10, typical)	≈ 90 ms
Mean over all 10 ticks	5.85 s
Peak process footprint (reported)	≈ 40.9 GB

Compiler breakdown. Instrumented `compile(...)` accumulates wall time inside each `expandNegacyclicToeplitz` call separately from the rest of lowering. On the release binary with `--compile-only`:

- **Host Toeplitz expand:** 21.59 s — one expansion per LUT cell, with no truth-table cache (4785 matrices; only 64 unique Yosys LUT strings exist in this netlist).
- **Graph build:** 3.87 s — input/DFF placeholders, LUT node attachment, enable/reset muxes, and output binds.

The debug boot’s much larger absolute setup time does not change the ranking: host expansion dominates; MPSGraph construction is secondary.

7.3 Correctness gates (pre-capstone)

Before scaling to PicoRV32, HELUT required:

- CPU $\leftrightarrow$ GPU bit-exact agreement on negacyclic matvec at torus corners;
- multi-tick DFF state retention without shape collapse;
- enable mux semantics so `$_DFFE_*` cells do not silently ignore clock enables.

Sync-reset (`$_SDFF*`) support and "x"-bit decoding were necessary for the PicoRV32 techmap; without them the compiler cannot close the netlist.

7.4 Memory scaling remark

A single LUT matrix is $\Theta(N^2)$ words. At $N = 1024$, $B = 1$, PicoRV32 already pressures tens of gigabytes of unified memory. Application sketches that raise B to 10^4 (batched search) imply $[B, N, N]$ broadcast intermediates on the order of tens of gigabytes *per reduction* and are memory-bound long before they are compute-bound [6]. HELUT therefore treats batch size as a first-class capacity knob, not a free ML-style hyperparameter.

8 Discussion

What the result shows. A commodity tensor API can host an *entire* synthesized soft CPU as one integer graph and step it under mock-encrypted control pins (ciphertext-structured vectors; Section 4.4). That is a systems existence proof about representation and plumbing, not a claim that FHE CPUs are practical on laptops today.

System	Exact $q=2^{32}$ Metal	Netlist clock	BK \$lut FHE	Diff. hardware
Concrete / tfhe-rs	–	compiler IR	yes	–
OpenFHE / HELib	–	circuit API	yes	–
HELUT boolean oracle	yes	yes	–	–
HELUT encrypted path	yes	yes	yes	–
TensorLUT (Pillar II)	–	Yosys INIT	–	yes

Table 4: Related-work feature matrix (scope, not throughput bake-off).

What would be required for “real” TFHE. Replace mock vectors with GLWE samples; insert key-switching / blind-rotation schedules instead of static Toeplitz feeds; manage noise and failure probability; and almost certainly abandon dense $N \times N$ materialization in favor of NTT or on-the-fly rotation. HELUT’s value in that future is the *netlist clocking* layer: once a PBS kernel exists as an `MPSGraph` subgraph, the same DFF loop applies.

Defensibility checklist.

1. Measurements in Table 3 are from a single instrumented run of the published harness; they are not averages over many devices.
2. “Encrypted” / “mock-encrypted” in the boot log means deterministic mock torus encodings without LWE noise (Section 4.4)—not cryptographic ciphertexts.
3. Steady-state latency excludes tick 1 JIT.
4. Peak memory includes host matrix buffers and Metal working sets; it is not a lower bound on an optimized implementation (e.g., matrix deduplication, streaming uploads).

9 Related Work

TFHE and FHEW established PBS as a Boolean-friendly FHE path [1, 2]. Industrial compilers (Concrete [4], HEIR/MLIR efforts) lower high-level programs to FHE ops; HELUT instead lowers *already-synthesized* gate netlists to a vendor ML graph. GPU FHE accelerators (cuFHE, and successors) optimize NTT-heavy kernels; HELUT intentionally uses the “wrong” dense embedding to stress-test whether `UInt32` tensor engines preserve modular semantics at all. PicoRV32 is a widely used size-optimized RISC-V core [5]; prior encrypted-CPU demos typically stop at ALUs or tiny FSMs rather than a full Yosys-mapped RV32 soft core on a phone/laptop NPU API.

10 Related work

11 Pillar II — Differentiable hardware

Objective. Let $w \in [0, 1]^{64L}$ be the concatenated INIT genome and $y(w)$ the multilinear TensorLUT forward pass. Soft crypto fitness and discreteness penalty are

$$F_{\text{crypto}}(w) = -\|y(w) - t\|_2^2, \quad (2)$$

$$\pi(w) = \sum_i w_i(1 - w_i), \quad (3)$$

$$F(w) = F_{\text{crypto}}(w) - \lambda \pi(w), \quad \lambda = \lambda_{\max}(p')^2 \geq 0. \quad (4)$$

Theorem 1 (TensorLUT continuous→discrete). *Under the hypotheses of `TensorLUTFormal.certificate()` (multilinear LUT, unfrozen $w_i \in [0, 1]$, involution sandwich on core INITs): $\pi \geq 0$ with equality*

on $\{0, 1\}^{64L}$; $F_{\text{crypto}} \leq 0$ with equality iff $y = t$; $F \leq F_{\text{crypto}}$ and increasing λ cannot improve F when $\pi > 0$; emitter $\mathbf{1}[w \geq \frac{1}{2}]$ is idempotent on binary INITs; stecker genotypes are partial involutions; freeze masks remove frozen blocks from π .

Machine-checked proof: `Sources/HELUTCore/TensorLUTFormal.swift` (`testTensorLUTFormalCertificate`).
Statement: `directives/tensorlut-theorem.md`. This is a *structural* theorem—not a U-534 break.

Theorem 2 (TensorLUT emitter / freeze corollary). *Under Theorem 1 hypotheses plus emit-via-E and mutatedPreserving for frozen stecker pairs: if $\pi(w) = 0$ then $E(w)$ recovers the binary INIT bits; every mutatedPreserving genotype retains frozen pairs and remains a partial involution (C25).*

Machine-checked: `TensorLUTFormal.corollaryCertificate()` (`testTensorLUTFormalCorollaryCertificate`).
Still not melt completeness.

Involution sandwich. Stecker genotypes are partial involutions (disjoint pairs) by construction (`SteckerInvolution.isValid`). The GA cannot propose a non-reciprocal map. Empirical grades: unmutated M4 baseline $F_{\text{crypto}} = 0$; full INIT / stecker-cone melts shatter under λ ; blind 3-pair rediscovery PASSED (campaign ledger).

12 Application gallery (nine slots)

Minimum disclosure bar: three runnable applications per pillar (`directives/application-gallery.md`).

Pillar	Slot	Reproduce / claim
I	Encrypted full-adder SING	C6, C20/C21
I	Tree / regex SING (demo N)	C6
I	Hardness + noisy-BK certs	C5, C22, C23, C26
II	M4 baseline emit	C8
II	Involution sandwich / formal	C9, C19, C25
II	Shatter vs hold (seminar)	empirical; not a decrypt (H6)
III	SoftBus reciprocity	C10, C24
III	Red battery grades	<code>enigma256_red.battery.sh</code>
III	Fail-closed NLFF harden	C24

13 Pillar III — Adversarial polymorphic ciphers

Objective. Enigma256 keeps the reciprocal rotor contract while deleting historical leaks, runs on SoftBus, and is graded under Red TensorLUT / KPA / `ent` pressure (empirical C10).

Theorem 3 (Enigma256 SoftBus reciprocity / fail-closed). *Under the hypotheses of `Enigma256Formal.certificate` (reciprocal scramble path; day-key involution builders; NLFF retaps do not rewrite frozen scramble): frozen scramble is a permutation and an involution; stream encrypt-then-decrypt recovers plaintext under identical keys; derived plugboard is fixed-point-free involution and un-reflector is an involution; `hardenedCubic()` rejects `coupledCubic6`.*

Machine-checked proof: `Sources/HELUTCore/Enigma256Formal.swift` (`testEnigma256FormalCertificate`).
Statement: `directives/enigma256-theorem.md`. This is a *structural SoftBus contract*—not IND-CPA and not a claim that Red pressure cannot force Blue generation rolls.

14 Limitations and Future Work

- **Lattice-estimator.** Calibrated core-SVP and Sage lattice-estimator disagree on 4/8 anchors beyond 16 bits (C23 / H1); production row agrees ($|\Delta|=4.5$). Do not quote calibrated 176 as estimator cost on every row.
- **Noisy BK.** Covering-gadget residuals are measured (C22). At product-shaped $N=1024$, inject $B=64$ is undecodable and $B=4$ on `.crypto` yields $\varepsilon \log_2 \approx -1$ (C26)—not 2^{-64} . Under $q=2^{32}$, exact public-MS covering exists only at $N \in \{8, 128\}$ (C27). Track B Metal SING at $N=128$ with inject $B=64$ PASSes (C28). Track A ($N=1024$ SING) still uses noiseless BK ($B_{bk}=0$, H4). $\ell=1$ `booleanPublicMS` cannot carry BK noise.
- **Encrypted scale.** Metal full_adder SING at $N=1024$ is measured (C20 / C21); sequential encrypted clocks and PicoRV FHE path remain open.
- **Oracle vs FHE.** Trivial/boolean Metal graphs are a shape laboratory, not the FHE claim.
- **Matrix deduplication.** Toeplitz expansion still benefits from truth-table caching on large netlists.
- **Side channels.** Metal/GPU power (bgpucap) is parked—not claimed.
- **Coverage.** Async resets, latches, and sequential encrypted clocks remain incomplete on the FHE path (combinational \$lut today).
- **Campaign.** P1030680 catalog / TensorLUT grades are parallel research —not a plaintext claim (H6 / N5–N7).

15 Conclusion

HELUT shows that Apple Silicon’s `MPSGraph` can carry an exact $\mathbb{Z}/2^{32}\mathbb{Z}$ negacyclic LUT datapath large enough to clock a Yosys-synthesized PicoRV32 core under a mock-encrypted reset sequence. The engineering lesson is prosaic and useful: once PBS is phrased as dense integer tensors, the rest of a soft CPU—wires, LUTs, enables, sync resets, and a host posedge loop—looks like ordinary graph IR work. The scientific lesson is cautionary: feasibility of the datapath is not feasibility of FHE. With that distinction stated plainly, HELUT stands as a defensible systems prototype and a measured baseline for denser, more cryptographic successors.

A Boot harness summary

The capstone executable loads `picorv32_netlist.json`, compiles with `YosysGraphCompiler`, locates the `reseth` placeholder, and runs ten ticks while rewriting the shared `reseth` `MTLBuffer` between `graph.run` calls (mock-encrypted 0 for three ticks, then mock-encrypted 1). All other primary inputs remain mock-encrypted 0. Reported compilation time wraps only `compile(...)`; per-tick times wrap only `graph.run`.

B Reproducibility

```
swift build
.build/debug/helut /path/to/picorv32_netlist.json
```

Requires macOS 14+, an Apple Silicon GPU with Metal, and sufficient unified memory (tens of GiB) for the PicoRV32 matrix working set.

References

- [1] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.
- [2] L. Ducas and D. Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In *EUROCRYPT*, 2015.
- [3] V. Lyubashevsky, C. Peikert, and O. Regev. On ideal lattices and learning with errors over rings. In *EUROCRYPT*, 2010.
- [4] Zama. Concrete: TFHE compiler and runtime. <https://github.com/zama-ai/concrete>, 2022–.
- [5] C. Wolf. PicoRV32 — a size-optimized RISC-V CPU. <https://github.com/YosysHQ/picorv32>.
- [6] HELUT Project. Application #2 note: batched encrypted search memory envelope ($B=10^4$, $N=1024$). Internal technical note, 2026.